



INSTITUTE OF BUSINESS MANAGEMENT

Final Year Project Management System
Software Requirements Specification (SRS)



Submitted by

Mahad Shahid 20221-32764
Muhammad Irtiza Zubair 20221-32618
Kazim Abbas 20221-32397
Muhammad Maaz Asim 20221-31865

College of Computer Science and Information Systems
Institute of Business Management (IoBM)

Date: 07/02/2026

Contents

1. Introduction.....	2
2. General Description.....	2
3. Functional Requirements.....	3
4. Interface Requirements.....	4
4.1 User Interfaces	4
4.2 Hardware Interfaces.....	5
4.3 Communications Interfaces	5
4.4 Software Interfaces	5
5. Performance Requirements.....	5
6. Other non-functional attributes	5
6.1 Security	5
6.2 Binary Compatibility	6
6.3 Reliability.....	6
6.4 Maintainability	6
6.5 Portability.....	6
7. Operational Scenarios.....	7
8. Preliminary Use Case Models and Sequence Diagrams	7
8.1 Use Case Model	7
8.2 Sequence Diagram.....	8

1. Introduction

1.1 Introduction

Introduction The purpose of this document is to define and describe the software requirements for the Final Year Project Management System (FYPMS). It details the system's functionality, constraints, interface requirements, and the interactions between stakeholders (Students, Advisors, Administrators) and the system. This document serves as a guideline for the development team and a validation reference for the client.

1.2 Scope of this Document

The primary users of this system are the students, faculty members (Advisors/Supervisors), and the administrative coordinators of the Computer Science Department at the Institute of Business Management (IoBM). The developers of the system are the FYP Group members. The system encompasses the entire lifecycle of a final year project, from initial group formation and proposal submission to the final defense and grading.

1.3 Overview

The product is a modern Web-Based Application utilizing a Decoupled Architecture (React.js Frontend + Python FastAPI Backend). It centralizes project management tasks and integrates Artificial Intelligence modules to solve specific administrative problems:

1. **Semantic Duplicate Detection:** Using NLP to prevent idea plagiarism.
2. **AI-Powered Advisor Matching:** Using keyword analysis to align student skills with faculty expertise.
3. **Automated Administrative Workflows:** For grading, scheduling, and reporting

1.4 Business Context

Currently, the department manages hundreds of student projects via manual spreadsheets, emails, and physical forms. This leads to data redundancy, lost paperwork, and inefficient supervisor allocation. The FYPMS aims to digitize this process, ensuring transparency, reducing administrative workload, and improving the quality of project supervision through intelligent matching.

2. General Description

2.1 Product Functions

The system acts as a central hub for all FYP activities. Key functions include:

- **User Management:** Role-based access for Admins, Advisors, and Students.
- **Project Initialization:** Group formation and proposal submission.
- **Intelligent Analysis:** Automatic checking of proposal uniqueness and supervisor compatibility.
- **Lifecycle Tracking:** Document uploads (SRS, SDS), progress monitoring, and defense scheduling.
- **Evaluation:** Digital grading sheets and automated result compilation

2.2 Similar System Information

Most existing university management systems (like Moodle or Blackboard) focus on general course management (LMS) rather than the specific workflows of a Final Year Project. While basic "Project Trackers" exist (e.g., Trello, Jira), they lack the academic context—specifically, they cannot detect plagiarized ideas against a university archive or

match students to supervisors based on research domains. Our system's AI Module provides this specific competitive advantage

2.3 User Characteristics

- **Students:** Undergraduate students with basic web literacy. They require a mobile-friendly interface to track deadlines and submit work.
- **Advisors (Faculty):** Professors who need a streamlined dashboard to review documents and grade students quickly without navigating complex menus.
- **Administrators:** Department coordinators who require high-level analytics and control over the entire batch's data.

2.4 User Problem Statement

The current manual system is slow, opaque, and error-prone.

- **Redundancy:** Students often submit project ideas that have already been implemented in previous years because there is no easy way to search the archive.
- **Mismatching:** Supervisors are often assigned groups outside their expertise due to manual scheduling constraints.
- **Communication Gaps:** Deadlines are missed due to a lack of centralized notifications.

2.5 User Objectives

- The system must automate the novelty check for proposals.
- The system must facilitate the fair and scientific allocation of supervisors.
- The system must generate automated reports (PDFs) to save administrative time.

2.6 General Constraints

- **Platform:** Must be a Web Application accessible via standard browsers (Chrome, Edge).
- **Backend:** Must be built in Python to support the required AI libraries (Sentence-Transformers, Scikit-Learn).
- **Connectivity:** Requires an internet connection to access the central server and database.

3. Functional Requirements

Requirement 3.1: Secure Role-Based Authentication The system shall allow users to log in with distinct roles (Student, Advisor, Admin) and direct them to their specific dashboards.

1. **Criticality:** Very High.

2. **Technical Risks:** Ensuring password security. We will use Bcrypt hashing to mitigate this.
3. **Dependencies:** Database availability.

Requirement 3.2: Semantic Duplicate Detection (Novelty Check) When a student submits a proposal, the system shall use Natural Language Processing (NLP) to compare the abstract against the archive of past projects. It must return a "Similarity Percentage".

- **Criticality:** High (Core "Standout" Feature).
- **Technical Risks:** The AI model might give false positives if the text is too generic.
- **Mitigation:** The system will flag high similarity (>80%) for human review rather than auto-rejecting.

Requirement 3.3: AI-Powered Advisor Matching The system shall parse student proposals to extract technical keywords (e.g., "IoT", "Blockchain") and match them against Advisor profiles to generate a ranked list of recommended supervisors.

- **Criticality:** Medium.
- **Technical Risks:** Lack of detailed Advisor profiles.
- **Dependencies:** Faculty must populate their "Research Interest" fields.

Requirement 3.4: Digital Evaluation & Grading Advisors and Committee members shall be able to input marks for defenses directly into the system, which will automatically calculate the GPA/Grade based on predefined formulas.

- **Criticality:** High.
- **Technical Risks:** Data consistency during concurrent access (multiple teachers grading at once).
- **Mitigation:** Database transactions and locking mechanisms.

Requirement 3.5: Automated Deadline Alerts The system shall run a background service that sends email or SMS notifications to students 3 days before a submission deadline.

- **Criticality:** Medium.
- **Technical Risks:** Email server (SMTP) failure

4. Interface Requirements

4.1 User Interfaces

4.1 User Interfaces

- **Frontend Framework:** React.js (Vite) with Tailwind CSS.
- **Design Philosophy:** Minimalist, clean design with a focus on usability.
- **Dashboards:**
 - *Student:* Progress bar, Submission upload box, Notifications.
 - *Admin:* Analytics charts (Pie/Bar graphs), User tables.

4.2 Hardware Interfaces

- **Server:** Cloud-based or Local Server with Python environment support.
- **Client:** Any internet-enabled device (Laptop, Tablet, Smartphone).

4.3 Communications Interfaces

- **Protocol:** HTTP/HTTPS (RESTful API).
- **Data Format:** JSON for data exchange between React (Frontend) and FastAPI (Backend).

4.4 Software Interfaces

- **Database:** PostgreSQL (Production) or SQLite (Development).
- **AI Engine:** Python libraries (`sentence-transformers`, `scikit-learn`).
- **PDF Engine:** `ReportLab` for generating official documents.

5. Performance Requirements

- **Response Time:** The web interface should load within 2 seconds.
- **AI Processing Speed:** The Duplicate Detection check should complete in under 5 seconds for a standard text input.
- **Concurrency:** The system should support at least 50 concurrent users (e.g., during proposal submission week) without crashing.
- **Uptime:** The system should aim for 99.9% availability during business hours

6. Other non-functional attributes

.

6.1 Security

- Passwords must never be stored in plain text (use Hashing).
- API endpoints must be protected using JWT (JSON Web Tokens) to prevent unauthorized access.

6.2 Binary Compatibility

The system is designed as a platform-independent web application compatible with any device running a modern browser (Chrome, Edge, Safari, Firefox), eliminating the need for specific OS or software installations. It ensures cross-platform functionality on Windows, macOS, Linux, and mobile devices (iOS/Android) while supporting data export to standard formats like PDF and Excel (.xlsx) for seamless integration with office workflows.

6.3 Reliability

The system should perform automated database backups daily to prevent data loss.

6.4 Maintainability

The code follows a Decoupled Architecture (Frontend separate from Backend), making it easy to update one side without breaking the other.

6.5 Portability

The web application is platform-independent and runs on Windows, macOS, and Linux clients via a web browser.

6.6 Extensibility

The system is built on a modular Decoupled Architecture, ensuring that new features (such as mobile app integration or advanced AI models) can be added without disrupting core functionality. Detailed API documentation (Swagger/OpenAPI) allows future developers with standard Python/React skills to extend the system easily.

6.7 Reusability

The backend code and AI modules are designed as reusable components. The Advisor Matching Engine and Duplicate Detection Service can be adapted for other university departments or research committees with minimal configuration changes.

6.8 Application Affinity/Compatibility

The application is fully compatible with modern web standards and integrates seamlessly with external tools. It supports data export to Microsoft Excel (.xlsx) and PDF formats, ensuring interoperability with existing university administrative workflows and office software.

6.9 Resource Utilization

The system is optimized for efficiency, running effectively on standard cloud hosting or local university servers (minimum 4GB RAM). It utilizes open-source technologies (Python, PostgreSQL, React), eliminating the need for expensive proprietary software licenses.

6.10 Serviceability

Maintenance is simplified through the use of industry-standard frameworks (FastAPI, React) and containerization (Docker). Any developer with basic knowledge of modern web development can perform updates, bug fixes, or database migrations using the provided documentation.

7. Operational Scenarios

Scenario A: Student Group Registration & Proposal

- **Actor:** Student
- **Action:** Logs in, creates a group by adding member IDs, and pastes the project abstract.
- **System Response:** The system validates member IDs, saves the group, runs the **Semantic Duplicate Check**, and displays the "Uniqueness Score." If unique, the proposal is forwarded to the Coordinator.

Scenario B: Admin Allocates Supervisor

- **Actor:** Administrator
- **Action:** Views the list of approved groups. Clicks "Suggest Supervisor."
- **System Response:** The AI analyzes the group's proposal keywords, checks Advisor loads, and displays a list: *"Recommended: Dr. Arif (95% Match - IoT Expert)"*. The Admin confirms the assignment.

Scenario C: Conducting a Defense

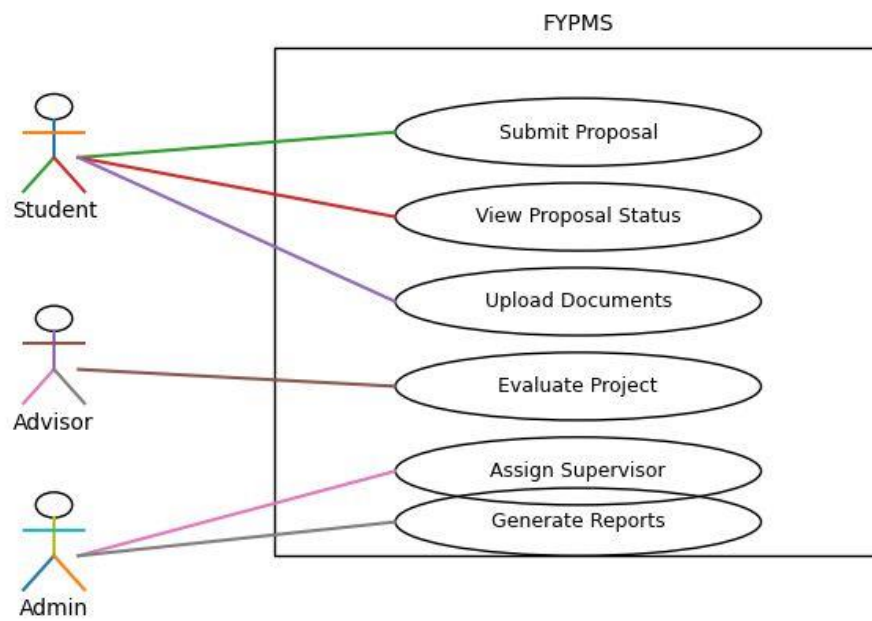
- **Actor:** Advisor
- **Action:** Opens the "Evaluation" tab during a student presentation. Enters marks for "Presentation Skills," "Implementation," and "Q&A."
- **System Response:** The system auto-calculates the total, saves the result, and updates the student's dashboard status to "Graded."

8. Preliminary Use Case Models and Sequence Diagrams

This section presents a list of the fundamental sequence diagrams and use cases that satisfy the system's requirements. The purpose is to provide an alternative, "structural" view of the requirements stated above and how they might be satisfied in the system.

8.1 Use Case Model

Use Case Diagram: FYPMS



8.2 Sequence Diagram

Sequence Diagram: Proposal Submission & Advisor Allocation

